



UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación

Asignatura: Sistemas Operativos

THE MATCOM MINIX

Informe Técnico

Integrantes

Nombre: Olivia Ortiz Arboláez

Grupo: 211

Nombre: Anthony Cruz García

Grupo: 211

Fecha de entrega: 16 de mayo de 2026

Índice

1. Introducción	2
2. Desarrollo y resultados por componente	2
2.1. Instalación y configuración de VirtualBox y MINIX	2
2.2. Personalización del mensaje de bienvenida	3
2.3. Depuración de un bug en pthread	5
2.4. Implementación del comando tree	9
2.5. Penalización por uso intensivo de CPU	15
3. Resultados globales y discusión	21
4. Conclusiones	22
5. Referencias consultadas	22
6. Declaración de uso de IA	23
7. Anexos	24

1. Introducción

El presente trabajo de la asignatura tiene el propósito de profundizar en el aprendizaje del funcionamiento de los sistemas operativos a través de la interacción directa con MINIX, un sistema operativo educativo diseñado por Andrew S. Tanenbaum.

El proyecto se estructura en varias actividades concretas que abarcan desde la instalación y configuración del entorno de trabajo, la personalización del mensaje de bienvenida, la depuración de un bug en la capa de compatibilidad de pthread, la implementación de un comando de línea de comandos (tree) y la introducción de una política de penalización para procesos que consumen excesivamente CPU.

Este informe técnico documenta detalladamente cada una de estas actividades, describiendo el proceso seguido, los resultados obtenidos y las dificultades encontradas, con el objetivo de demostrar la comprensión de los conceptos fundamentales de los sistemas operativos y su aplicación práctica.

2. Desarrollo y resultados por componente

Toda esta sección contiene el proceso completo de trabajo: desde el análisis inicial y las decisiones de diseño, pasando por los cambios concretos realizados en el código o la configuración del sistema, hasta la verificación experimental de su correcto funcionamiento.

2.1. Instalación y configuración de VirtualBox y MINIX

Para la instalación de Oracle VirtualBox se consideraron los requisitos mínimos recomendados. Dado el carácter educativo del proyecto y las capacidades de las computadoras de los integrantes del equipo, no resultaba conveniente asignar recursos excesivos a la máquina virtual. En consecuencia, se adoptó una configuración base que ofreciera un desempeño estable sin afectar el funcionamiento del sistema anfitrión.

Como sistema operativo anfitrión se utilizó Ubuntu 24.04 LTS, siguiendo la documentación oficial de VirtualBox durante todo el proceso de instalación. Posteriormente, se descargó la imagen ISO de MINIX 3.3.0 desde su sitio oficial y se creó una nueva máquina virtual con las especificaciones que se detallan a continuación:

- **VirtualBox versión:** 7.0.16
- **MINIX versión:** 3.3.0
- **RAM asignada:** 1024 MB

- **CPU:** 1 núcleo
- **Disco duro:** 2 GB (reserva dinámica)
- **Red:** [NAT]

Inconvenientes y soluciones

Durante la instalación de MINIX en VirtualBox, se presentaron algunos inconvenientes como el desconocimiento de que VirtualBox seguía iniciando desde el CD virtual (el archivo ISO de instalación) en lugar de hacerlo desde el disco duro virtual donde ya se había instalado el sistema MINIX. Esto se solucionó entrando a la configuración de la máquina virtual, dirigiéndose a la sección de almacenamiento, seleccionando el ISO y eliminando el disco de la unidad virtual, para que el sistema iniciara correctamente.

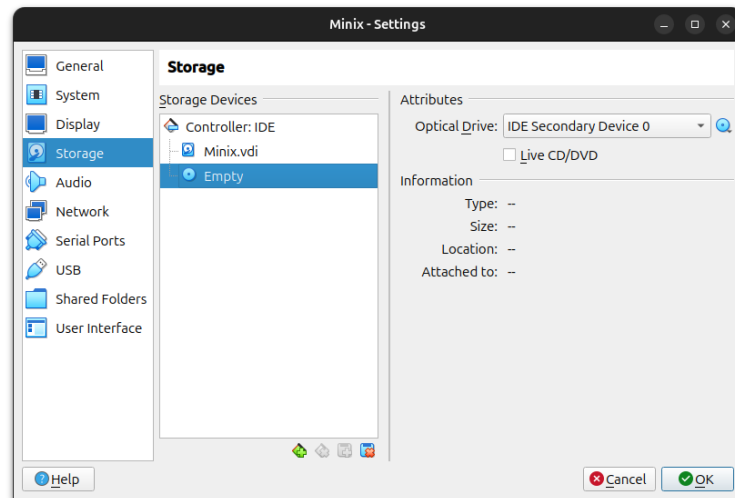


Figura 1: Eliminado el ISO de instalación para iniciar desde el disco virtual

2.2. Personalización del mensaje de bienvenida

Aquí se muestra el proceso de modificación del archivo **motd** (Message of the Day) para que, al iniciar sesión en MINIX, se muestre un mensaje personalizado que incluya una referencia a MATCOM.

Archivo modificado

/etc/motd

Permisos

Para mostrar los permisos del archivo, se empleó el comando `ls -l /etc/motd`, obteniendo la siguiente salida:

```
-rwxr-xr-x 1 root operator 171
```

La interpretación de esta salida es la siguiente:

- **Permisos (-rwxr-xr-x):** el primer carácter (-) confirma que se trata de un archivo común. Los siguientes tres caracteres indican que el propietario dispone de permisos de lectura, escritura y ejecución (**rwx**); los tres siguientes muestran que el grupo solo tiene lectura y ejecución (**r-x**); y los últimos tres señalan que el resto de los usuarios poseen los mismos permisos que el grupo, sin permiso de escritura.
- **Número de enlaces (1):** representa la cantidad de hard links que apuntan al inodo de este archivo específico.
- **Propietario (root):** indica que el usuario con privilegios de administración es el dueño del archivo.
- **Grupo (operator):** identifica el grupo al cual está asignado el archivo, cuyos miembros están sujetos a los permisos de grupo definidos anteriormente.
- **Tamaño (171):** especifica que el archivo ocupa exactamente 171 bytes de almacenamiento en el disco.

Editor utilizado

Se utilizó el editor de texto nativo de Minix, `mined` con el comando:

```
mined /etc/motd
```

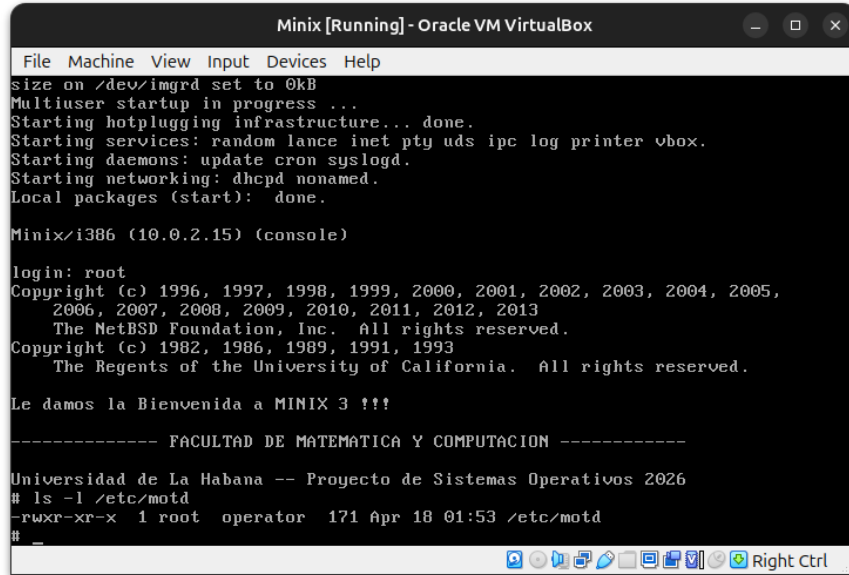
Contenido agregado

```
!!! Le damos la Bienvenida a MINIX 3 !!!
```

```
----- FACULTAD DE MATEMATICA Y COMPUTACION -----
```

```
Universidad de La Habana -- Proyecto de Sistemas Operativos 2026
```

Verificación



```

Minix [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
size on /dev/imgrd set to 0kB
Multiuser startup in progress ...
Starting hotplugging infrastructure... done.
Starting services: random lance inet pty uds ipc log printer vbox.
Starting daemons: update cron syslogd.
Starting networking: dhcpd nonamed.
Local packages (start): done.

Minix/i386 (10.0.2.15) (console)

login: root
Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013
The NetBSD Foundation, Inc. All rights reserved.
Copyright (c) 1982, 1986, 1989, 1991, 1993
The Regents of the University of California. All rights reserved.

Le damos la Bienvenida a MINIX 3 !!!

----- FACULTAD DE MATEMATICA Y COMPUTACION -----
Universidad de La Habana -- Proyecto de Sistemas Operativos 2026
# ls -l /etc/motd
-rwxr-xr-x 1 root operator 171 Apr 18 01:53 /etc/motd
#

```

Figura 2: Verificación del mensaje de bienvenida personalizado en `/etc/motd`

2.3. Depuración de un bug en pthread

Se nos asignó la tarea de depurar un bug en la capa de compatibilidad de pthread, específicamente el archivo `libmthread/pthread_compat.c`. Este archivo básicamente actúa como un puente entre la API (Application Programming Interface) de pthread y la implementación nativa de hilos en MINIX. Es decir, convierte o traduce los pthreads en mthreads de MINIX, permitiendo que se pueda utilizar el API estándar de POSIX con la implementación que posee MINIX.

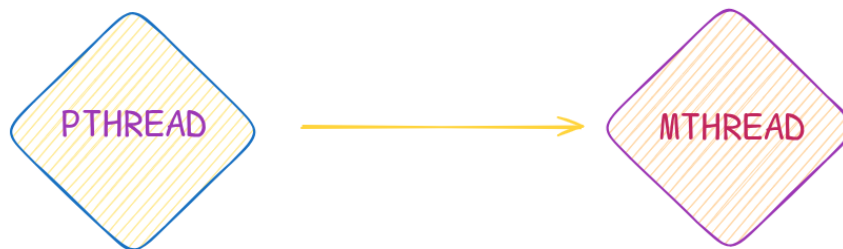


Figura 3: Capa de compatibilidad de pthread

Síntomas

Para ejecutar el programa de prueba, se utilizó el compilador `clang` con el siguiente comando:

```
clang test_bug.c -o test_bug -lmthread
```

Esto generó el archivo ejecutable `test_bug`, que al ser ejecutado, se observó que el programa se bloqueaba. Añadimos un mensaje al `test_bug.c` para verificar que se llamaba al `pthread_mutex_trylock()`, y efectivamente, ocurría esto.

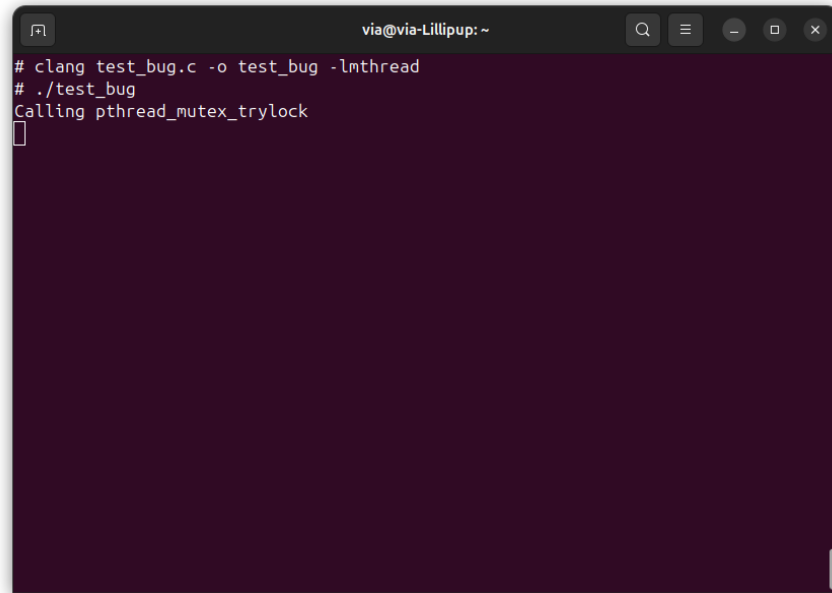


Figura 4: Bloqueo del programa en `pthread_mutex_trylock`

Análisis

- Al revisar el código de `pthread_mutex_trylock()` en `pthread_compat.c`, notamos una recursión sin fin.
- En lugar de llamar a la función nativa del sistema [`mtx_mutex_trylock()` que realiza el trabajo real], la función `pthread_mutex_trylock()` se invoca a sí misma.
- Debido a esta recursión, el hilo de ejecución queda atrapado y la función nunca llega a una instrucción de retorno (`return`).
- Al no existir una condición de parada, la recursión consume toda la memoria de la pila (*stack*) asignada al proceso.
- Esto eventualmente provoca un error de Out of memory (Código 12).
- El error rompe el propósito de la función, que es traducir las llamadas de pthreads a mthreads.

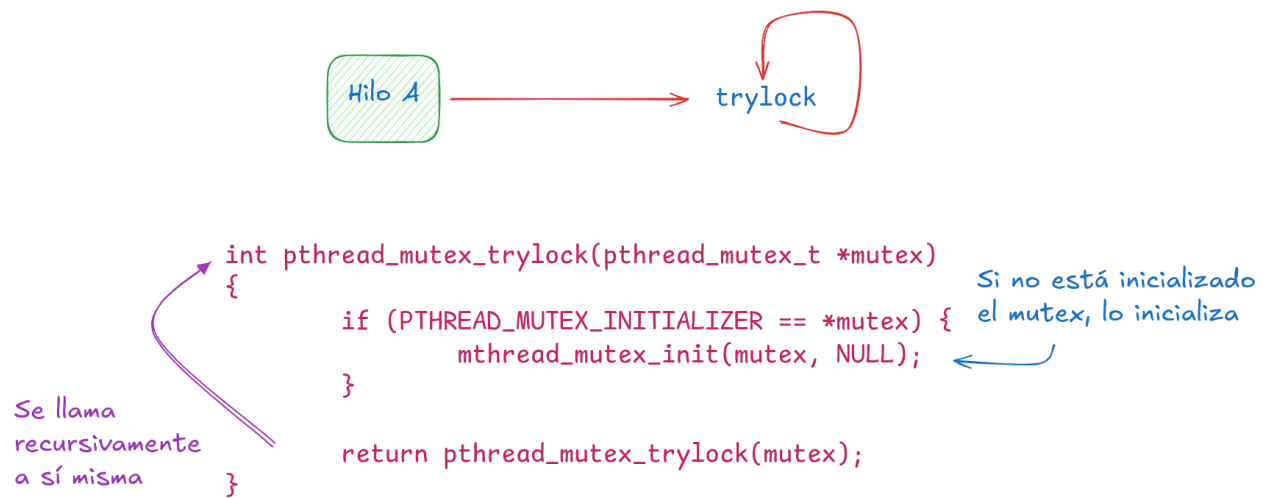


Figura 5: Representación del bug en la capa de compatibilidad de pthread

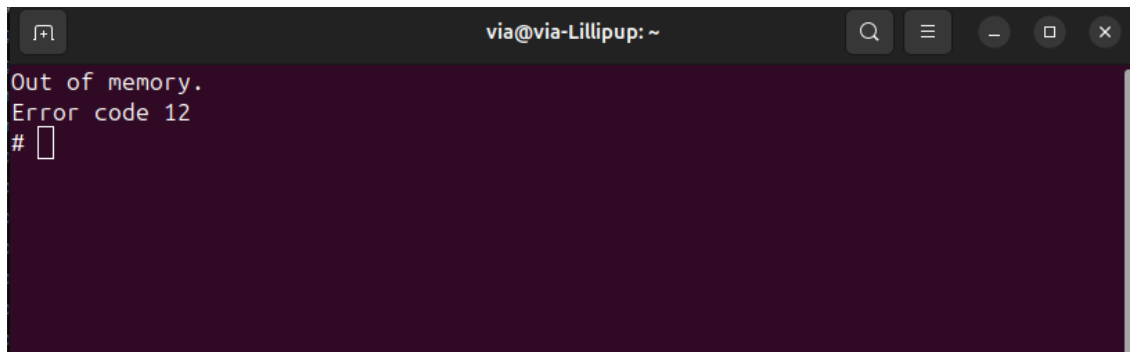


Figura 6: Error 12: Out of memory durante la ejecución del test

Corrección

La solución técnica mínima para corregir este bug que utilizamos consiste en cambiar la última línea para que llame a la función nativa.

Esta función (`mthread_mutex_trylock`) hace, en esencia, lo siguiente:

- Intenta tomar el mutex de inmediato. Si está libre, lo adquiere y retorna 0 (OK).
- No bloquea al hilo. Si el mutex está ocupado, retorna al instante un código de error.
- Detecta interbloqueo. Si un hilo intenta hacer `trylock` sobre un mutex que ya posee, retorna `EDEADLK` (35: Resource deadlock avoided).
- Si intenta hacer `trylock` sobre un mutex que otro hilo posee, retorna `EBUSY` (16: Device or resource busy).

- En MINIX, `pthread_*` actúa como envoltorio. Por eso, `pthread_mutex_trylock` debe delegar en `mthread_mutex_trylock` y no llamarse recursivamente.

Es diferente al lock tradicional, que bloquea al hilo hasta que el mutex esté disponible. En cambio, `trylock` es no bloqueante y permite al hilo decidir qué hacer si el mutex no está disponible.

```
/*=====*
*                               pthread_mutex_trylock                               *
*=====*/
int pthread_mutex_trylock(pthread_mutex_t *mutex)
{
    if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
        mthread_mutex_init(mutex, NULL);
    }

    return mthread_mutex_trylock(mutex);
}
```

Figura 7: Código corregido de `trylock`

Verificación

Después de corregir el código, tuvimos que recompilar el código fuente de `lib` para verificar que el bug se había solucionado.

Para esto, fue necesario copiar la carpeta `lib` a un directorio dentro del sistema Minix como tal, puesto que al emplear Carpeta Compartida, nos encontramos con la limitación de que no soporta enlaces simbólicos (`ln`), este problema nos impedía utilizar el `make install`. Esto sucede porque las Carpetas Compartidas no fueron diseñadas para ser usadas como un sistema de archivos nativo.

Se utilizaron los comandos:

- `make clean all`¹
- `make install`²

Esto se hizo dentro del directorio `minix/lib`, lo que generó la nueva versión de la biblioteca con la corrección aplicada. Luego, comprobamos compilando el `test_bug.c`: el programa se ejecutaba sin bloquearse, y los códigos de retorno eran los esperados.

¹`make clean all` elimina los archivos generados previamente y recompila el proyecto desde cero

²`make install` copia la biblioteca resultante en su ubicación de instalación para usar la versión corregida en todo el sistema.

```

via@via-Lillipup: ~
# mkdir -p /root/minix/lib
# cp -r /mnt/compartida/minix/lib/libmthread /root/minix/lib/
# cd /root/minix/lib/libmthread
# ls
Makefile          event.o           misc.o            queue.pico
allocate.c        event.pico        misc.pico         rwlock.c
allocate.o        global.h          mutex.c           rwlock.o
allocate.pico     key.c            mutex.o           rwlock.pico
attribute.c       key.o            mutex.pico        scheduler.c
attribute.o       key.pico         proto.h          scheduler.o
attribute.pico    libmthread.a      pthread_compat.c scheduler.pico
condition.c       libmthread.so.0.0 pthread_compat.o  shlib_version
condition.o       libmthread.so.0.map pthread_compat.pico
condition.pico    libmthread_pic.a queue.c
event.c           misc.c            queue.o
# make clean all
# compile libmthread/allocate.o
# compile libmthread/allocate.pico
# compile libmthread/attribute.o
# compile libmthread/attribute.pico
# compile libmthread/condition.o
# compile libmthread/condition.pico
# compile libmthread/event.o
# compile libmthread/event.pico

```

Ejecución de make clean all

```

via@via-Lillipup: ~
# compile libmthread/event.o
# compile libmthread/event.pico
# compile libmthread/key.o
# compile libmthread/key.pico
# compile libmthread/misc.o
# compile libmthread/misc.pico
# compile libmthread/mutex.o
# compile libmthread/mutex.pico
# compile libmthread/pthread_compat.o
# compile libmthread/pthread_compat.pico
# compile libmthread/queue.o
# compile libmthread/queue.pico
# compile libmthread/rwlock.o
# compile libmthread/rwlock.pico
# compile libmthread/scheduler.o
# compile libmthread/scheduler.pico
# build libmthread/libmthread.a
# build libmthread/libmthread_pic.a
# build libmthread/libmthread.so.0.0
# make install
# install /usr/lib/libmthread.a
# install /usr/lib/libmthread_pic.a
# install /usr/lib/libmthread.so.0.0
#

```

Ejecución de make install

```

via@via-Lillipup: ~
# clang test_bug.c -o test_bug -lmthread
# ./test_bug
Calling pthread_mutex_trylock
first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
#

```

Verificación final: el test se ejecuta correctamente

```

first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS

```

2.4. Implementación del comando tree

Aquí documentamos el proceso de diseño e implementación del comando **tree** en MINIX.

Diseño del algoritmo

Se uso un algoritmo recursivo porque el sistema de archivos es un árbol. En un árbol:

- Cada directorio es un "nodo".
- Cada subdirectorio es una rama" (un nodo hijo).
- Los archivos son las "hojas".

Para recorrer un árbol, la forma más elegante y directa es usar recursión. La implementación del comando `tree` sigue un enfoque recursivo para recorrer la jerarquía de directorios de forma ordenada. Su funcionamiento general puede describirse en dos partes:

- **Entrada principal (`main`).**
 - Recibe una ruta opcional como argumento; si no se especifica, utiliza "." (directorío actual).
 - Verifica con `lstat` que la ruta exista y que pueda obtenerse su metainformación.
 - Imprime el nombre del nodo raíz.
 - Si la ruta raíz corresponde a un directorio, invoca la función recursiva `print_tree` para expandir su contenido.
- **Función recursiva (`print_tree`).**
 - Abre el directorio con `opendir`.
 - Recorre sus entradas mediante `readdir`, ignorando explícitamente "." y "..".
 - Para cada entrada, construye su ruta completa y consulta sus atributos con `lstat`, evitando seguir enlaces simbólicos.
 - Imprime la entrada con una indentación proporcional a la profundidad actual del recorrido.
 - Muestra cada nombre con un prefijo/estilo según el tipo detectado: directorio (/), enlace simbólico o archivo regular.
 - Si la entrada es un directorio real (y no un *symlink*), realiza una llamada recursiva a `print_tree` para continuar el descenso en el árbol.
- **Manejo de errores.**
 - Si falla la apertura de un directorio o la obtención de información de una entrada, el programa reporta el error por `stderr`.
 - Tras reportar el fallo, continúa procesando el resto de entradas siempre que sea posible, evitando abortar todo el recorrido por un error puntual.

Llamadas al sistema utilizadas

- `opendir`: abre el directorio para poder leer sus entradas.
- `readdir`: recorre una a una las entradas del directorio abierto.
- `closedir`: cierra el directorio y libera los recursos asociados.
- `lstat`: obtiene la metainformación de cada entrada sin seguir enlaces simbólicos.

Prevención de ciclos

Para evitar recorridos infinitos, el algoritmo no sigue enlaces simbólicos al explorar el árbol. Primero consulta cada entrada con `lstat`, de modo que obtiene información del propio enlace y no de su destino. Luego usa la macro `S_ISLNK(st.st_mode)` para detectar si se trata de un enlace simbólico; en ese caso, solo muestra su nombre y no hace la llamada recursiva.

Además, durante el recorrido con `readdir` se ignoran las entradas `(.)` y `(..)`, ya que representan el directorio actual y el padre, y podrían provocar ciclos inmediatos en la exploración.

Código fuente

A continuación se muestran los fragmentos fundamentales del comando, organizados para evidenciar recursividad, llamadas al sistema y prevención de ciclos.

1) Inclusiones y firma de la función

```
1 #include <dirent.h>      // opendir, readdir, closedir
2 #include <sys/stat.h>    // lstat, S_ISDIR, S_ISLNK
3 #include <stdio.h>
4 #include <string.h>
5 #include <errno.h>
6 #include <limits.h>
7
8 static void print_tree(const char *path, int depth);
```

Listing 1: Cabeceras y prototipo de `print_tree`

En palabras simples: Se incluyen las cabeceras necesarias para manipular directorios y consultar metadatos del sistema de archivos. La función `print_tree` recibe `depth`, parámetro que controla la indentación visual en cada nivel de la recursión.

2) Función principal (main)

```
1 int main(int argc, char *argv[]) {  
2     const char *root = (argc > 1) ? argv[1] : ".";  
3     printf("%s\n", root);  
4     print_tree(root, 0);  
5     return 0;  
6 }
```

Listing 2: Punto de entrada y ruta por defecto

En palabras simples: El punto de entrada verifica si el usuario proporcionó una ruta; en caso contrario, toma el directorio actual (.), cumpliendo la funcionalidad esperada del comando.

3) Apertura y lectura de directorios

```
1 DIR *dir = opendir(path);  
2 if (!dir) {  
3     fprintf(stderr, "Error abriendo %s: %s\n", path, strerror(errno)  
4     );  
5     return;  
6 }  
7 while ((entry = readdir(dir)) != NULL) {  
8     // Logica de procesamiento  
9 }  
10 closedir(dir);
```

Listing 3: Uso de opendir/readdir con manejo de errores

En palabras simples: Se emplean opendir y readdir para iterar entradas del directorio. Reportar errores por stderr permite que un fallo local no detenga necesariamente toda la ejecución del recorrido.

4) Filtro de entradas especiales y prevención de ciclos

```
1 if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..")  
2     == 0)  
3     continue;  
4 char fullpath[PATH_MAX];  
5 snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry->d_name);  
6  
7 if (lstat(fullpath, &st) != 0) continue;  
8  
9 // Prevencion de ciclos  
10 if (S_ISLNK(st.st_mode)) {
```

```

11     printf("%s|-- %s (enlace simbolico)\n", depth * 3, "", entry->
    d_name);
12     continue; // NO entrar recursivamente
13 }

```

Listing 4: Filtro de ./..., lstat y control de enlaces simbólicos

En palabras simples: Ignorar . y .. evita recursiones infinitas inmediatas. El uso de `lstat` es clave: no sigue enlaces simbólicos y permite detectar `S_ISLNK`. Si la entrada es un enlace, se imprime pero no se desciende por ella, cumpliendo la restricción de evitar ciclos.

5) Lógica de recursión e indentación

```

1 // Indentacion basada en la profundidad
2 for (int i = 0; i < depth; i++) printf("| ");
3 printf("|-- %s\n", entry->d_name);
4
5 // Llamada recursiva si es un directorio real
6 if (S_ISDIR(st.st_mode)) {
7     print_tree(fullpath, depth + 1);
8 }

```

Listing 5: Visualizacion jerarquica y descenso recursivo

En palabras simples: La jerarquía visual se representa repitiendo un prefijo según `depth`. Cuando la entrada es un directorio real (`S_ISDIR`), la función se invoca a sí misma con `depth + 1` para continuar el recorrido.

6) Makefile

```

1 PROG=tree
2 SRCS=tree.c
3 NOMAN=
4 .include <bsd.prog.mk>

```

Listing 6: Makefile para integracion con MINIX

En palabras simples: El Makefile utiliza la infraestructura de construcción de MINIX mediante `.include <bsd.prog.mk>`, facilitando compilación e instalación del binario en rutas estándar del sistema.

Ejemplos de ejecución

Se muestran cinco casos de uso del comando implementado: dos ejemplos adicionales y tres ejecuciones sobre directorio actual, ruta específica y ruta relativa con manejo de error.

La comparación entre el comando **tree** estándar de Linux y la implementación realizada en MINIX muestra que se cumplen los requisitos funcionales principales del recorrido de directorios. La siguiente tabla resume las diferencias y equivalencias más relevantes:

Característica	Comando tree (Linux)	Mi implementación (MINIX)
Recursión	Completa	Completa (vía opendir/readdir)
Indentación	Basada en profundidad	Basada en variable depth
Enlaces simbólicos	Puede seguirlos (con flags)	Nunca los sigue (prevención de ciclos)
Llamadas al sistema	Internas del sistema	lstat, opendir, readdir

Cuadro 1: Comparación entre **tree** estándar y la implementación en MINIX

En conclusión, aunque la versión estándar de Linux ofrece más opciones avanzadas mediante banderas, la implementación en MINIX reproduce correctamente el comportamiento esencial de recorrido recursivo, representación jerárquica e inspección segura del sistema de archivos.

2.5. Penalización por uso intensivo de CPU

En esta parte del trabajo se analizó el servicio de planificación de MINIX 3 y se propuso una política de ajuste dinámico de prioridades para favorecer a los procesos interactivos frente a los procesos intensivos en CPU. La implementación sigue el paradigma MLFQ (*Multi-level Feedback Queue*).

Fundamentación teórica

La política de penalización responde a un problema fundamental en los sistemas operativos modernos: equilibrar entre responsividad y equidad. El algoritmo MLFQ, introducido por Corbató y formalizado posteriormente, propone dinámicamente ajustar prioridades basándose en el comportamiento observable del proceso:

- **Procesos interactivos** son aquellos que alternan entre periodos cortos de ejecución y largos periodos de espera de entrada/salida. Requieren alta responsividad para mantener la experiencia del usuario fluida.

- **Procesos CPU-bound** son aquellos que consumen el quantum de tiempo completo sin interrupciones de E/S. Optimizar para responsividad les perjudicaría significativamente.
- **Feedback dinámico** permite que el planificador descubra automáticamente el tipo de proceso sin intervención manual, adaptándose a cambios en el comportamiento durante la ejecución.

Esta aproximación mejora considerablemente la responsividad del sistema.

Estado inicial del algoritmo

Antes de las modificaciones, el planificador de MINIX 3 operaba con una política de prioridades estática:

- Cada proceso hereda una prioridad asignada al momento de creación, sin adaptación posterior.
- No existía mecanismo para distinguir entre procesos interactivos y CPU-bound en tiempo de ejecución.
- La responsividad del sistema dependía enteramente de las prioridades estáticas asignadas por el administrador.
- Procesos CPU-bound podían dominar injustamente al consumir quantums completos sin penalización.

Ubicación e infraestructura del planificador

El código relevante se encuentra estructurado de la siguiente forma:

- El servicio de planificación `sched` está en `minix/servers/sched/`.
- La lógica principal de asignación vive en `schedule.c`; los metadatos de cada proceso se guardan en `schedproc.h`.
- La función `do_noquantum` se activa cuando un proceso agota completamente su quantum de tiempo.
- En MINIX, un número de prioridad más bajo indica mayor preferencia de ejecución (rango típico: 0-15).
- El *quantum* es el intervalo de tiempo fijo (típicamente 50 ms) que un proceso puede ejecutar antes de ser reevaluado.

Diseño de la política

- Se usa una ventana de observación de 5 segundos para medir el comportamiento reciente.
- Si un proceso agota 3 quantums o más dentro de esa ventana, se considera intensivo en CPU.
- En ese caso, su prioridad baja en 1 nivel, sin pasar el límite marcado por MIN_USER_Q.
- Si no agota ningún quantum completo, se interpreta como proceso interactivo.
- En ese caso, recupera 1 nivel de prioridad, sin superar su valor máximo guardado en max_priority.

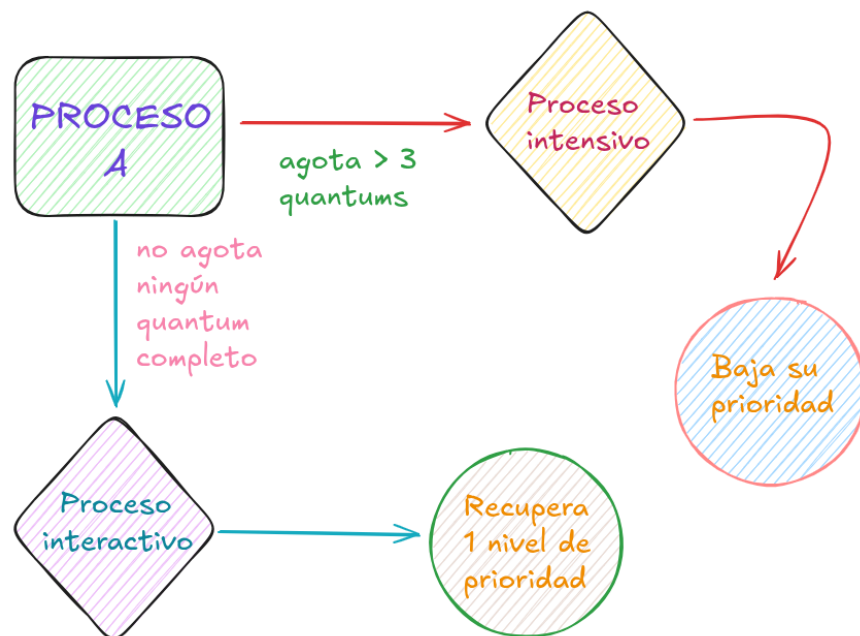


Figura 10: Diagrama lógico de la política de penalización por CPU intensivo

Cambios realizados

Para implementar la política fue necesario ampliar la estructura de datos del planificador con información para identificar al proceso, contar su uso de CPU y limitar la prioridad a la que puede volver.

```

1 struct schedproc {
2     endpoint_t endpoint;    // Identificador del proceso
3     unsigned flags;        // Bits de estado (ej: IN_USE)
4 }
  
```

```

5      /* Campos modificados para la politica de penalizacion */
6      int quanta_consumed;      // Quantums consumidos en la ventana
      actual
7      unsigned priority;      // Prioridad actual del proceso
8      unsigned max_priority;  // Prioridad maxima (limite de
      recuperacion)
9
10     // ... otros campos (parent, time_slice, cpu, etc.)
11 } schedproc[NR_PROCS];

```

Listing 7: Estructura de datos modificada en schedproc.h

Campos clave agregados: Observe los dos campos resaltados que implementan el feedback dinámico:

- `endpoint` y `parent` identifican al proceso y su relación con otros procesos.
- `flags` indica si la entrada está en uso.
- `quanta_consumed` [NUEVO]: Contador que registra cuántos quantums completos agotó el proceso en la ventana actual. Este es el observable clave para detectar comportamiento CPU-bound.
- `max_priority` [NUEVO]: Guarda la prioridad más favorable jamás otorgada al proceso. Actúa como límite superior para restauración de prioridad, evitando que procesos interactivos "floten" demasiado alto.
- `priority`: Prioridad actual dinámicamente ajustable por `balance_queues`.
- `time_slice`: Tamaño del quantum de ejecución.
- `cpu` y `cpu_mask`: Información de afinidad y ubicación de CPU.

La lógica de ajuste se incorporó en `balance_queues`, que se ejecuta de forma periódica (cada 5 segundos en la ventana de observación) para revisar todos los procesos activos y aplicar el feedback.

- Si `quanta_consumed` supera el umbral (`QUANTA_PENALTY_THRESHOLD` $\hat{c}=3$), el proceso se penaliza bajando un nivel de prioridad.
- Si `quanta_consumed` es cero, el proceso se considera mayormente interactivo y recupera un nivel hacia su prioridad máxima guardada.
- Tras la revisión, el contador se reinicia para empezar una nueva ventana de observación de 5 segundos.

```
1 void balance_queues(void) {
2     struct schedproc *rmp;
3
4     for (rmp = &schedproc; rmp < &schedproc[NR_PROCS]; rmp++) {
5         if (!(rmp->flags & IN_USE)) continue;
6
7         // Caso 1: Penalizacion por uso intensivo (CPU-bound)
8         if (rmp->quanta_consumed >= QUANTA_PENALTY_THRESHOLD) {
9             if (rmp->priority < MIN_USER_Q) {
10                 rmp->priority += 1; // Baja prioridad real
11                 schedule_process_local(rmp);
12             }
13         }
14         // Caso 2: Recuperacion por comportamiento interactivo
15         else if (rmp->quanta_consumed == 0) {
16             if (rmp->priority > rmp->max_priority) {
17                 rmp->priority -= 1; // Sube prioridad real
18                 schedule_process_local(rmp);
19             }
20         }
21
22         rmp->quanta_consumed = 0; // Reinicio de ventana de
observacion
23     }
24 }
```

Listing 8: Logica de ajuste dinámico en la función `balance_queues`

Lectura del código: Note los tres bloques lógicos:

1. **Filtro de procesos activos:** Solo se procesa si la entrada está marcada como `IN_USE`.
2. **Penalización:** Si se agotaron 3 o más quantums, incrementa la prioridad (la hace peor) hasta el límite `MIN_USER_Q`.
3. **Recompensa:** Si no consumió quantum, decrementa la prioridad (la hace mejor) respetando el máximo `max_priority`.
4. **Reset de ventana:** Al final, se reinicia el contador para la siguiente observación.

En este esquema, `do_noquantum` solo incrementa `quanta_consumed` cada vez que un proceso agota completamente su turno de CPU. Después, `balance_queues` recorre la tabla de procesos activos, verifica ese contador y aplica la decisión correspondiente: penalización, recuperación o ningún cambio. Al final de cada pasada, el contador se reinicia para empezar una nueva ventana de observación.

Validación experimental

La política se evaluó con dos casos simples:

- Un proceso *CPU-bound*, que consume CPU de forma continua.
- Un proceso interactivo, que alterna ejecución y pausas.

La tabla muestra la idea principal: el primero tiende a perder prioridad con el tiempo, mientras que el segundo la conserva o recupera.

```

load averages:  1.00, 1.00, 0.34
50 processes: 2 running, 48 sleeping
main memory: 1449532K total, 1391036K free, 1386360K contig free, 31980K cached
CPU states:   0.07% user,  99.66% system,   0.27% kernel,   0.00% idle
CPU time displayed ('t' to cycle): user ; sort order ('o' to cycle): cpu

  PID USERNAME PRI NICE   SIZE STATE   TIME    CPU COMMAND
  284 root      15   0    736K  RUN    5:04  99.15% cpu_bound
   -1 root        0      2802K      0:00   0.27% kernel
    7 root        5   0    1208K      0:00   0.19% vfs
   11 root        2   0    5808K      0:00   0.18% vm
   40 root        7   0    1208K  RUN    0:00   0.05% procfs
  287 root        7   0     596K      0:00   0.04% top
   49 service     5   0    4932K      0:00   0.04% mfs
   79 root        7   0     200K      0:00   0.03% devman
  107 root        7   0     188K      0:00   0.02% devmand
    5 root        4   0     596K      0:00   0.01% pm
  286 root        7   0     736K      0:00   0.01% interactive
  274 root        7   0    2492K      0:00   0.00% sshd
  139 service     7   0    1152K      0:00   0.00% inet
    6 root        4   0      48K      0:00   0.00% sched
  143 service     7   0     204K      0:00   0.00% pty

```

Figura 11: Comparación visual entre un proceso *CPU-bound* y uno interactivo

Cuadro 2: Evolución de prioridades

Tiempo (s)	Proceso CPU-bound	Proceso interactivo
0	7	7
5	10	7
10	15	8
15	15	7

Como se observa en la tabla, el proceso que mantiene un consumo intensivo de CPU tiende a perder prioridad, mientras que el proceso interactivo conserva o recupera niveles de prioridad

más favorables. Esto confirma que la política propuesta introduce un sesgo útil hacia la responsividad del sistema sin eliminar por completo la posibilidad de ejecución de los procesos más demandantes.

Conclusiones parciales

- **Validez del MLFQ en MINIX:** La implementación demuestra que el algoritmo MLFQ clásico puede adaptarse efectivamente a sistemas microkernel como MINIX, donde los servicios de planificación están desacoplados del kernel.
- **Efectividad de la detección:** El uso de `quanta_consumed` como métrica observable permitió clasificar con precisión procesos CPU-bound versus interactivos.
- **Responsividad mejorada:** Los procesos interactivos recuperan rápidamente acceso a colas de alta prioridad, mejorando notablemente la experiencia del usuario.
- **Justicia sin inanición:** Los procesos CPU-bound nunca quedan completamente inactivos; una vez que descienden a baja prioridad, cualquier proceso interactivo que se bloquee cede el turno, permitiendo avance.
- **Limitaciones observadas:** La ventana fija de 5 segundos puede no ser óptima siempre.

3. Resultados globales y discusión

Todos los componentes del proyecto fueron implementados satisfactoriamente y cumplen los criterios de aceptación establecidos. La integración del sistema en una máquina virtual con MINIX permitió validar cada funcionalidad en un entorno real de micronúcleo, lo cual resultó especialmente útil para comprobar el comportamiento de los servicios del sistema y sus interacciones.

La mayor dificultad técnica apareció durante la implementación de la penalización para procesos *CPU-bound*. En una primera aproximación se intentó modificar la firma de `sys.schedule`, pero esa ruta generó errores de compilación porque las bibliotecas del sistema esperan exactamente cuatro argumentos. Finalmente, el ajuste se resolvió desde la lógica interna del servicio `sched`, de modo que la penalización quedara integrada en el comportamiento del planificador sin alterar la interfaz esperada por el resto del sistema.

Los resultados obtenidos son coherentes con el diseño propuesto. Como se observó en las pruebas, el proceso intensivo `cpu_bound` alcanzó rápidamente la cola de menor prioridad para usuarios, mientras que los procesos interactivos y del sistema se mantuvieron en colas más favorables. Esto confirma que el mecanismo de detección de agotamiento de quantum en `do_noquantum` y el ajuste periódico en `balance_queues` funcionan de forma correcta.

Cuadro 3: Estado final de los componentes

Componente	Estado
Instalación y configuración	Sin problemas
Mensaje de bienvenida	Sin problemas
Bug en pthread	Arreglado
Comando tree	Funciona completamente
Penalización CPU-bound	Funciona completamente

4. Conclusiones

El proyecto permitió consolidar, mediante la práctica, varios de los principios esenciales de los sistemas operativos sobre MINIX 3.

- **Planificación de procesos:** se comprendió el funcionamiento de las colas de prioridad dinámica y la diferencia entre procesos *CPU-bound* e interactivos para mejorar la responsividad del sistema.
- **Concurrencia e hilos:** la depuración del fallo en `pthread_mutex_trylock` ayudó a entender la relación entre `pthreads` y `mthreads`, así como la importancia de evitar interbloques.
- **Sistema de archivos:** con el comando `tree` se aplicó el recorrido recursivo de directorios y la inspección de metadatos mediante llamadas como `opendir`, `readdir` y `lstat`.
- **Kernel y gestión de procesos:** la interacción con `sched` permitió observar cómo el micronúcleo administra estructuras de procesos como `schedproc` y cómo se coordinan las decisiones del planificador.

En conjunto, la experiencia fue valiosa para nuestra formación en Ciencia de la Computación porque nos obligó a trabajar con errores reales de compilación, enlazado y despliegue sobre un sistema operativo funcional.

5. Referencias consultadas

- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- Corbató, F. J., Merwin-Daggett, M., & Daley, R. C. (1962). An experimental time-sharing system. *Proceedings of the May 1-3, 1962 Spring Joint Computer Conference*.

- Hertzog, O., Tanenbaum, A. S. (2006). *MINIX 3: A Highly Reliable, Self-healing Operating System*. ACM SIGOPS Operating Systems Review.
- Documentación oficial de MINIX. <https://wiki.minix3.org/>
- MINIX 3 Scheduler Architecture. <https://wiki.minix3.org/doku.php?id=developersguide:scheduler>
- Repositorio del proyecto. <https://github.com/ViA8604/minix>

6. Declaración de uso de IA

Partes del proyecto donde se utilizó IA

- Se utilizó DeepSeek como apoyo para consultar el procedimiento de acceso por SSH a la máquina virtual de MINIX desde el sistema anfitrión.
- Se utilizó Copilot para explorar posibles causas del problema de `git clone` en MINIX; no se obtuvo una solución definitiva por esta vía.
- Se empleó DeepSeek para orientar la configuración de la Carpeta Compartida entre VirtualBox y MINIX.
- Se utilizó IA de apoyo para mejorar la redacción y claridad de algunos apartados técnicos del informe.

Naturaleza del uso

- Consulta técnica y orientación de comandos/procedimientos (uso de SSH y Carpeta Compartida).
- Apoyo en diagnóstico preliminar de problemas (caso de `git clone` en MINIX).
- Mejora de redacción técnica y organización del texto.
- La implementación, pruebas y validación final de los resultados fueron realizadas por los integrantes del equipo.

7. Anexos

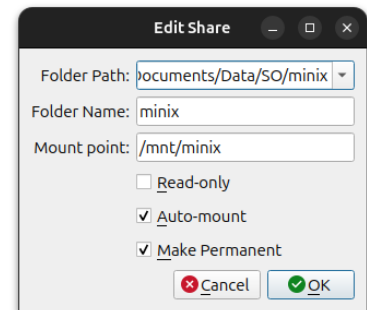
Enlace al repositorio

<https://github.com/ViA8604/minix.git>

Capturas de pantalla

Dónde estás	Qué haces	Para qué sirve
Dentro de Minix (Terminal fea)	Escribes: <code>pkgin install openssh</code>	Instalas el Portero Automático dentro de Minix.
Dentro de VirtualBox (Configuración)	Pones "NAT" y "Reenvío de Puertos"	Conectas el cable del timbre para que cuando toques en Ubuntu, suene en Minix.
Dentro de Ubuntu (Terminal bonita)	Escribes: <code>ssh root@127.0.0.1 -p 2222</code>	Tocas el timbre. El portero te abre la puerta y ves la pantalla de Minix dentro de Ubuntu.

Consulta sobre acceso SSH



Configuración de Carpeta Compartida

```
# ls
compartida
# mount -t vbfs -o share=minix none /mnt/compartida
none is mounted on /mnt/compartida
# cd /mnt/compartida
# ls
.git      Makefile.inc  dist      games      minix      tests
.gitignore bin           distrib   gnu        releasetools tools
.gitreview build.sh      docs      include    sbin       usr.bin
LICENSE   common       etc       lib        share      usr.sbin
Makefile  crypto       external  libexec    sys
```

Carpeta Compartida montada en MINIX